Configure the Kafka Client & Gateway

Configure the Gravitee Kafka Gateway and Kafka client to proxy a Kafka cluster, including TLS, broker mapping, mTLS authentication, and producing and consuming messages.

Overview

To use Gravitee to proxy a Kafka cluster, configure the Gravitee Kafka Gateway and a Kafka client.

Configure the Kafka Gateway

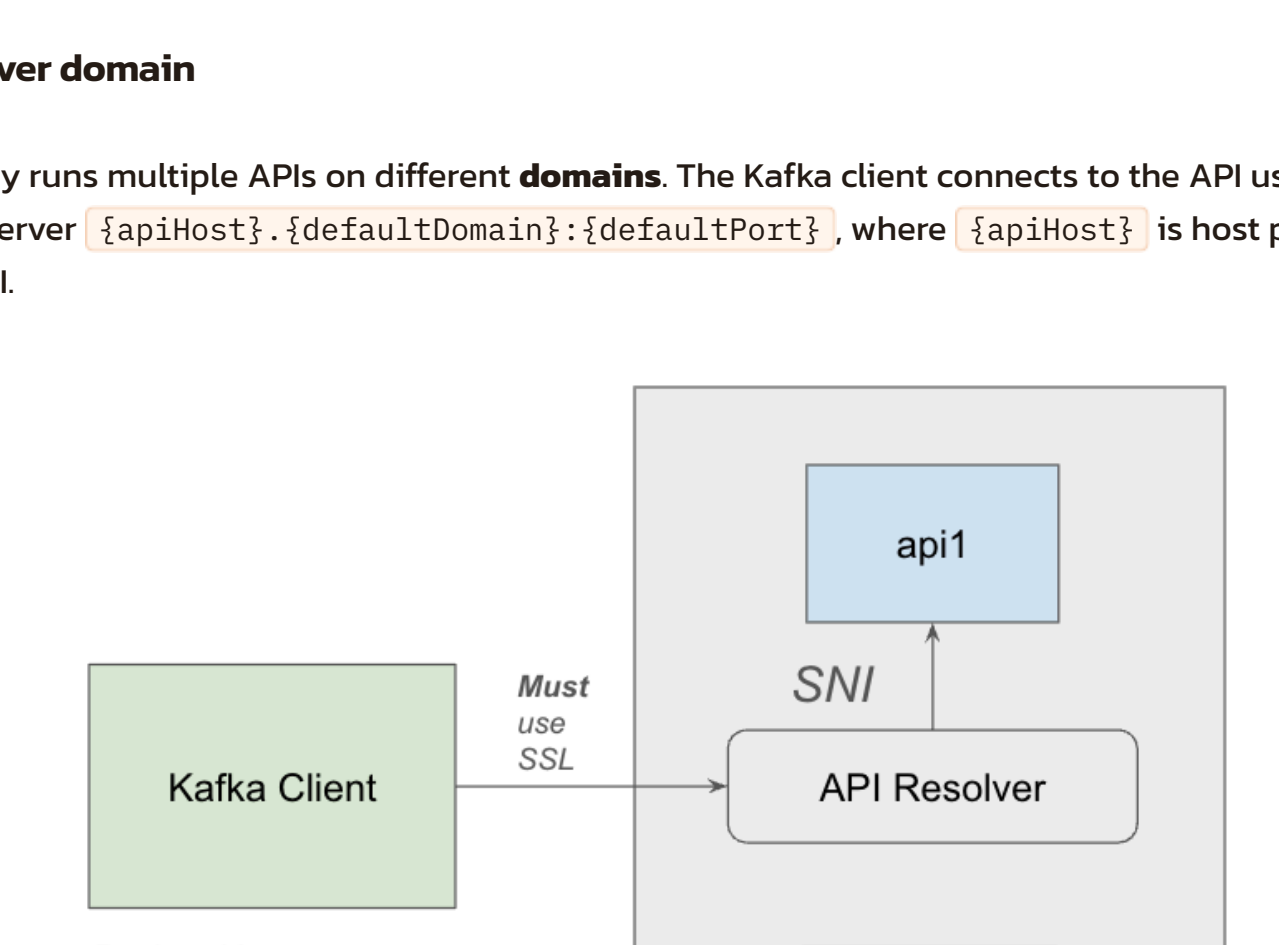
Running the Kafka Gateway requires an Enterprise license with the Kafka Gateway feature included. This does not come by default with a Universe license; it must be purchased separately from Gravitee.

To run the Kafka Gateway, enable the Gateway server in `gravitee.yml`. The full example of the configuration is defined below. The baseline required configuration is:

```
kafka:
  enabled: true

  routingMode: host # default is host. Supported values: host, port. For port-based routing,
  # Routing Host Mode
  routingHostMode:
    brokerPrefix: "broker-" # default is broker-
    domainSeparator: "-" # Used to separate broker's name from api & defaultDomain. Default i

    # The default domain where the Kafka APIs are exposed. ex: `myapi` will be exposed as `my
    defaultDomain: "mycompany.org" # Should set according to the public wildcard DNS/Certific
    defaultPort: 9092 # Default public port for Kafka APIs. Default is 9092
```

Bootstrap server domain

- The Gateway runs multiple APIs on different **domains**. The Kafka client connects to the API using the bootstrap server `{apiHost}.{defaultDomain}:{defaultPort}`, where `{apiHost}` is host prefix defined for each API.


api1
SNI
Must
use
SSL
Kafka Client
API Resolver
api2
Declared bootstrap server:
api1.mycompany.org:9092
Security Protocol:
SSL or SASL_SSL
Kafka Gateway Server

The Kafka client routes to the correct API through the gateway using SNI routing.

- To route to the correct API, the Gateway uses SNI routing ↗, which is part of the TLS protocol. Consequently, all client connections **must** happen over TLS, with at least `security.protocol=SSL` set in the Kafka client configuration.
- The client **must** trust the certificate provided by the Gateway. To handle the variable host in the proxy bootstrap server URL, you likely need to request a wildcard SAN to use as the certificate presented by the Gateway.
- With the default configuration, you need a wildcard DNS entry so that you don't need a new DNS entry for every API. In this example, the DNS and wildcard certificate are for `*.mycompany.org`.

What if I have restrictions on the domains I can use?

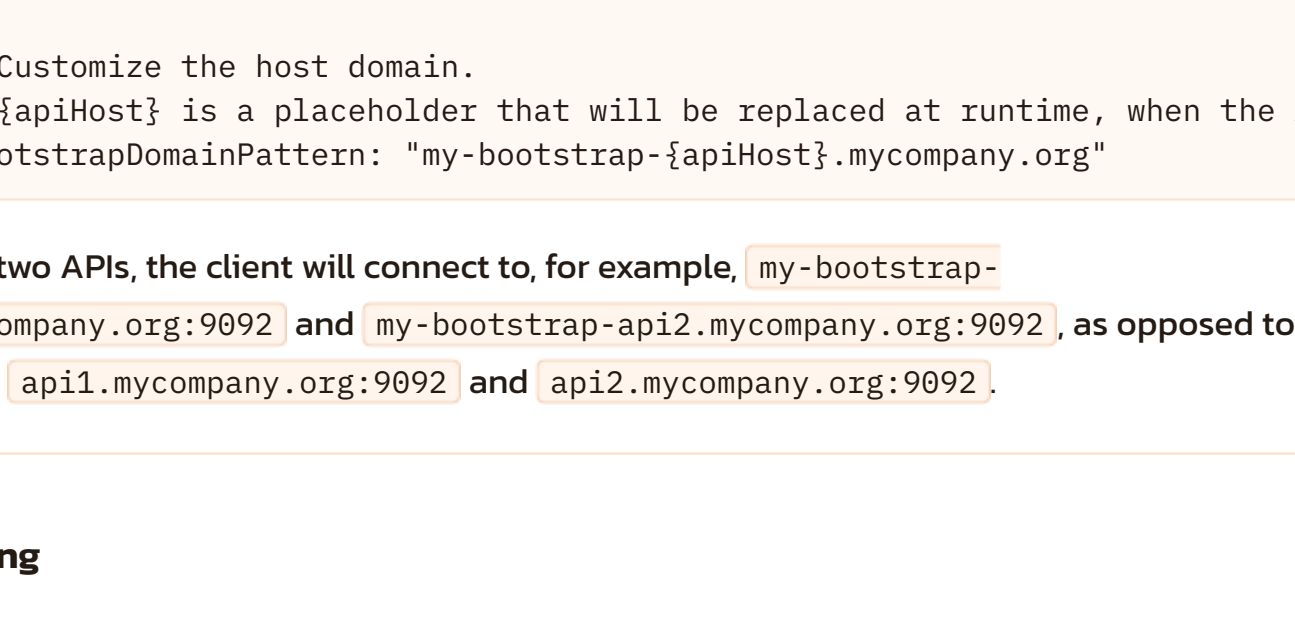
If you have restrictions on the domain names you can use for APIs, you can override the default hostname by updating the Gateway configuration. For example, instead of `{apiHost}.{defaultDomain}` as the hostname, you can set the pattern to `my-bootstrap-{apiHost}.mycompany.org` by configuring the variables below:

```
kafka:
  enabled: true

  routingMode: host # default is host. Only host is supported for now.
  # Routing Host Mode
  routingHostMode:
    brokerPrefix: "broker-" # default is broker-
    domainSeparator: "-" # Used to separate broker's name from api & defaultDomain.

    # The default domain where the Kafka APIs are exposed. ex: `myapi` will be expos
    defaultDomain: "mycompany.org" # Should set according to the public wildcard DNS
    defaultPort: 9092 # Default public port for Kafka APIs. Default is 9092

    # Customize the host domain.
    # {apiHost} is a placeholder that will be replaced at runtime, when the API is d
    bootstrapDomainPattern: "my-bootstrap-{apiHost}.mycompany.org"
```

Then, for two APIs, the client will connect to, for example, `my-bootstrap-api1.mycompany.org:9092` and `my-bootstrap-api2.mycompany.org:9092`, as opposed to the default of `api1.mycompany.org:9092` and `api2.mycompany.org:9092`.

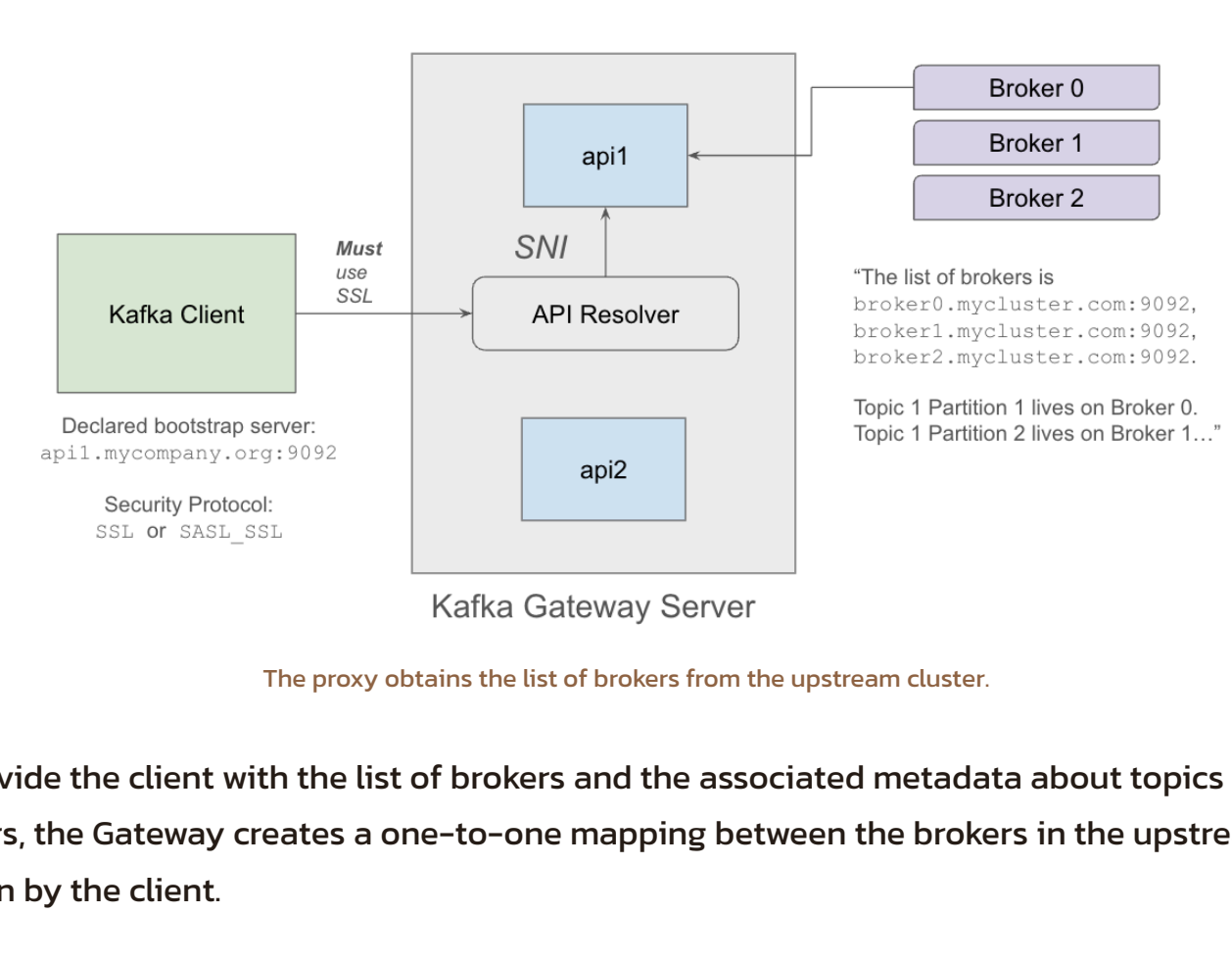
Broker mapping

After the Kafka client connects to the API, the Gateway (acting as the bootstrap server) returns the list of brokers in the upstream cluster.


api1
Must
use
SSL
SNI
Kafka Client
API Resolver
api2
Declared bootstrap server:
api1.mycompany.org:9092
Security Protocol:
SSL or SASL_SSL
Kafka Gateway Server
Broker 0
Broker 1
Broker 2
"The list of brokers is
broker0.mycluster.com:9092,
broker1.mycluster.com:9092,
broker2.mycluster.com:9092.
Topic 1 Partition 1 lives on Broker 0. Topic 1 Partition 2 lives on Broker 1…"

The proxy obtains the list of brokers from the upstream cluster.

To properly provide the client with the list of brokers and the associated metadata about topics and partitions on those brokers, the Gateway creates a one-to-one mapping between the brokers in the upstream cluster and the brokers seen by the client.


"The list of brokers is
broker-0-api1.mycompany.com:9092,
broker-1-api1.mycompany.com:9092,
broker-2-api1.mycompany.com:9092.
Topic 1 Partition 1 lives on Broker 0. Topic 1 Partition 2 lives on Broker 1…"
api1
Must
use
SSL
SNI
Kafka Client
API Resolver
api2
Declared bootstrap server:
api1.mycompany.org:9092
Security Protocol:
SSL or SASL_SSL
Kafka Gateway Server

The gateway returns the list of brokers back to the client, rewritten to use the gateway hostname.

The mapping combines the `brokerPrefix`, `brokerSeparator`, and `defaultDomain` variables, along with the API host prefix. The Kafka client must be able to route to `{brokerPrefix}-{brokerId}-{apiHost}.{defaultDomain}`, for as many brokers as there are in the Kafka cluster. Again, a wildcard DNS entry is the preferred way to do this.

What if I have restrictions on the domains I can use?

If you have restrictions on the domain names you can use for APIs, you can override the broker domain pattern, as described in What if I have restrictions on the domains I can use?. The configuration will then be as follows (with `brokerDomainPattern` being the relevant option):

```
kafka:
  enabled: true

  routingMode: host # default is host. Only host is supported for now.
  # Routing Host Mode
  routingHostMode:
    brokerPrefix: "broker-" # default is broker-
    domainSeparator: "-" # Used to separate broker's name from api & defaultDomain.

    # The default domain where the Kafka APIs are exposed. ex: `myapi` will be expos
    defaultDomain: "mycompany.org" # Should set according to the public wildcard DNS
    defaultPort: 9092 # Default public port for Kafka APIs. Default is 9092

    # Customize the host domain.
    # {apiHost} is a placeholder that will be replaced at runtime, when the API is d
    # {brokerId} is a placeholder that stand for the broker id
    bootstrapDomainPattern: "my-bootstrap-{apiHost}.mycompany.org"
    brokerDomainPattern: "{apiHost}-broker-{brokerId}-test.mycompany.org"
```

With this, if there are three brokers in the upstream cluster, the client must be able to route to `api1-broker-0-test.mycompany.org`, `api1-broker-0-test.mycompany.org`, and `api1-broker-0-test.mycompany.org`, along with `my-bootstrap-api1.mycompany.org`.

What if I don't have a valid DNS entry?

If you do not have a valid DNS entry for your Gateway because, for example, you're running the Gateway on `localhost`, then you may need to update your `/etc/hosts` file.

If you are running the Gateway locally in Docker, and setting the `defaultDomain` to `kafka.local`, you can update your `/etc/hosts` file with the following entries:

```
127.0.0.1    localhost kafka.local api1.kafka.local
::1          localhost broker-0-api1.kafka.local broker-1-api1.kafka.local broker-2-
127.0.0.1    localhost broker-0-api1.kafka.local broker-1-api1.kafka.local broker-2-
```

To add more APIs, you will need to add another API host to the first line and two more entries for each API to the IPs `::1` and `127.0.0.1`. With two APIs, this becomes:

```
127.0.0.1    localhost kafka.local api1.kafka.local api2.kafka.local
::1          localhost broker-0-api1.kafka.local broker-1-api1.kafka.local broker-2-
127.0.0.1    localhost broker-0-api1.kafka.local broker-1-api1.kafka.local broker-2-
::1          localhost broker-0-api2.kafka.local broker-1-api2.kafka.local broker-2-
127.0.0.1    localhost broker-0-api2.kafka.local broker-1-api2.kafka.local broker-2-
```

Port allocation for port-based routing

When using port-based routing, each plan requires three port values:

- **Bootstrap port**: The entry point clients use to connect (range: 1024–65535).
- **Broker range start**: The beginning of the contiguous port range allocated for backend Kafka brokers (range: 1024–65535).
- **Broker range end**: The end of the contiguous port range allocated for backend Kafka brokers (range: 1024–65535).

The broker range must not overlap with the bootstrap port or with port allocations from other plans in the same environment.

Port conflict detection

The Gateway enforces four conflict rules within each environment to prevent port allocation conflicts:

- Broker ranges from different plans cannot overlap.
- A plan's bootstrap port cannot fall within another plan's broker range.
- A plan's broker range cannot contain another plan's bootstrap port.
- Bootstrap ports must be unique across all plans.

Conflict detection runs before plan creation or update and blocks operations that would violate these rules.

Configure mTLS authentication

All Kafka plans already use TLS for transport encryption by default. The configuration below is **only** required when using an mTLS plan, which adds client certificate authentication on top of standard TLS.

To require client certificate authentication for Native Kafka APIs, add the following SSL properties to `gravitee.yml`. This is an infrastructure-level configuration that applies globally to all Native Kafka APIs deployed on the gateway:

Property	Description	Example
`kafka.ssl.clientAuth`	Require client certificate authentication	`required`
`kafka.ssl.truststore.type`	Gateway truststore type for verifying client certificates	`jks`
`kafka.ssl.truststore.password`	Gateway truststore password	`gravitee`
`kafka.ssl.truststore.path`	Path to gateway truststore containing CA that signed client certificates	`/path/to/server.truststore.jks`
`kafka.ssl.keystore.type`	Gateway keystore type	`jks`
`kafka.ssl.keystore.password`	Gateway keystore password	`gravitee`
`kafka.ssl.keystore.path`	Path to gateway keystore	`/path/to/server.keystore.jks`

The gateway truststore must contain the CA certificate that signed the client certificates. The gateway keystore contains the gateway's identity certificate.

To reject connections from clients whose certificates have been revoked, add a `kafka.ssl.crl` block to `gravitee.yml`. The CRL options mirror the HTTP server's CRL options — see Reject revoked client certificates with a CRL for the full reference and behavior.

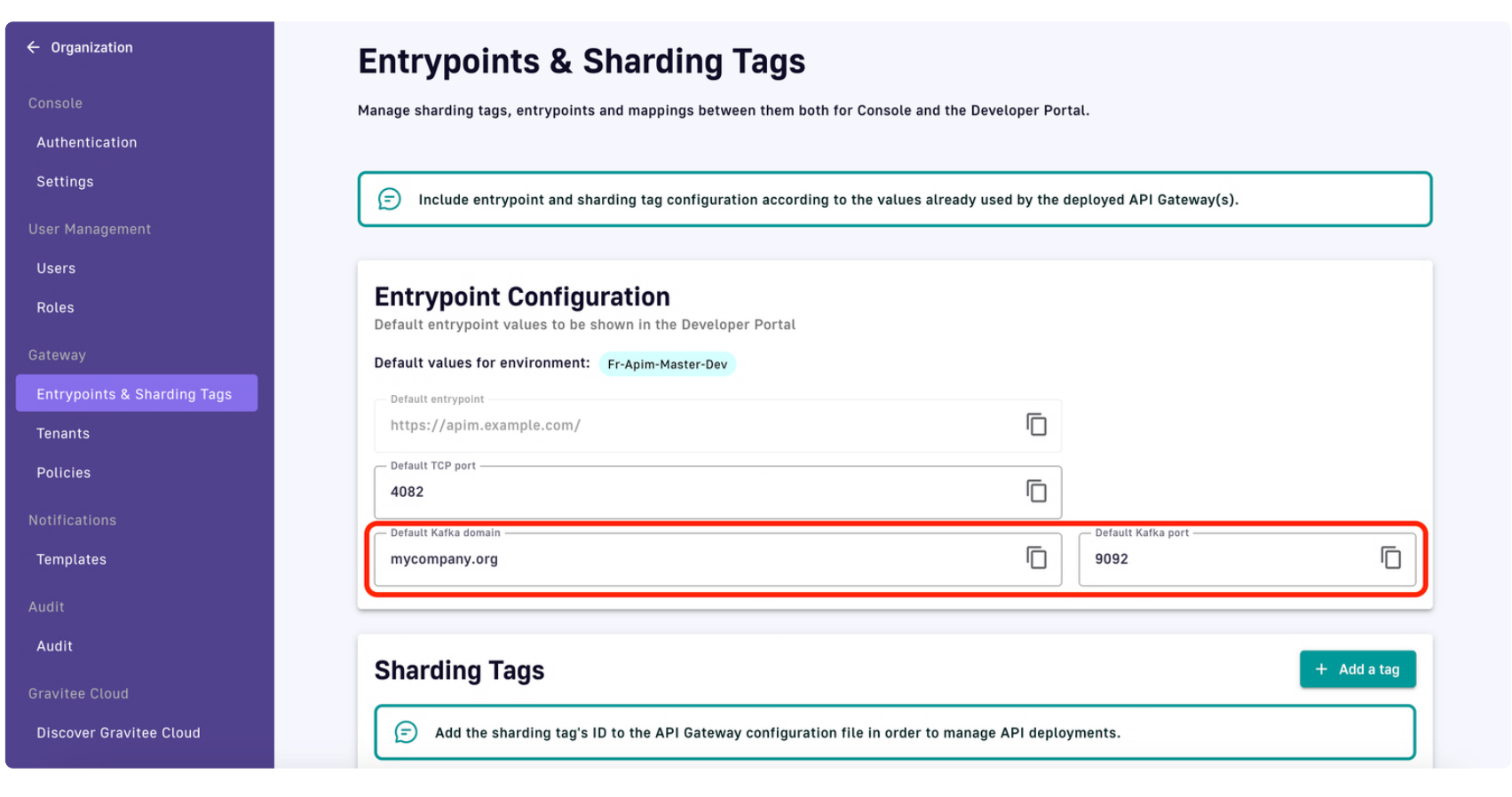
```
kafka:
  ssl:
    crl:
      path: /path/to/crl
      watch: true
```

Define the default entrypoint configuration

By default, clients talk to Kafka APIs by setting the bootstrap server as `{apiHost}.{defaultDomain}:{defaultPort}`. This is set in `gravitee.yml`, but for convenience, when developing APIs in the UI, you can set the default values appended to the hostname. You can also leave this value blank and respecify the full hostname in the API.

To configure the APIM Console to use the Kafka domain and port values for your Organization:

1. Log in to your Management Console.
2. Select **Organization**.
3. Select **Entrypoints & Sharding Tags**.
4. In the **Entrypoint Configuration** section, confirm that the **Default Kafka domain** and **Default Kafka port** values match those of your Kafka API.


Organization
Console
Authentication
Settings
User Management
Users
Roles
Gateway
Entrypoints & Sharding Tags
Tenants
Policies
Notifications
Templates
Audit
Audit
Gravitee Cloud
Discover Gravitee Cloud
Entrypoints & Sharding Tags
Manage sharding tags, entrypoints and mappings between them both for Console and the Developer Portal.
Include entrypoint and sharding tag configuration according to the values already used by the deployed API Gateway(s).
Entrypoint Configuration
Default entrypoint values to be shown in the Developer Portal
Sharding Tags
+ Add a tag
Add the sharding tag's ID to the API Gateway configuration file in order to manage API deployments.

Default values for environment: Fr-Apim-Master-Dev

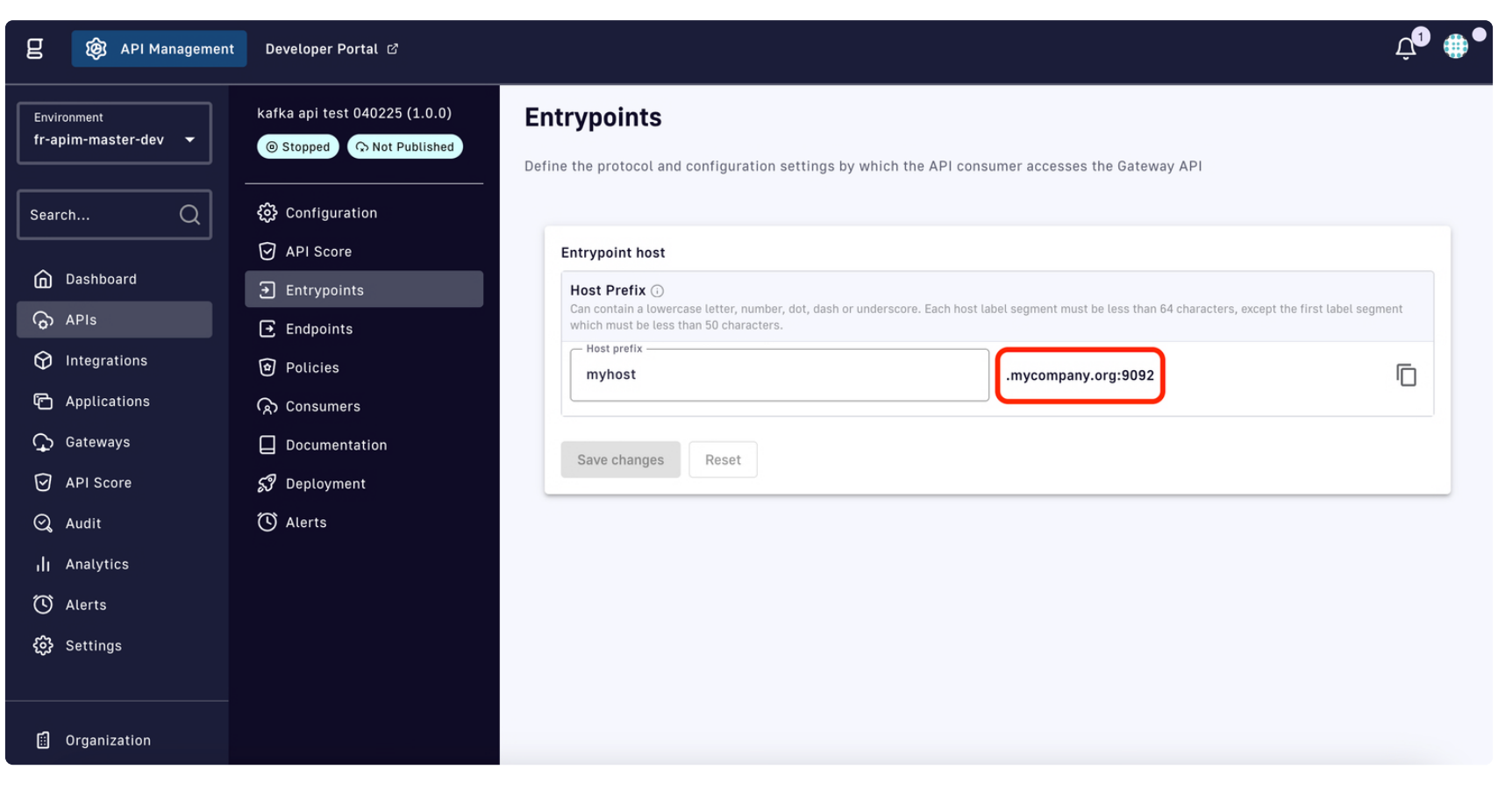
Default entrypoint: https://apim.example.com/

Default TCP port: 4082

Default Kafka domain: mycompany.org

Default Kafka port: 9092

This value is then displayed on the entrypoint page of your APIs.


API Management
Developer Portal
Environment
Search...
Dashboard
APIs
Integrations
Applications
Gateways
API Score
Audit
Analytics
Alerts
Settings
Organization
Stopped
Not Published
Configuration
API Score
Entrypoints
Endpoints
Policies
Consumers
Documentation
Deployment
Alerts
Entrypoints
Define the protocol and configuration settings by which the API consumer accesses the Gateway API
Entrypoint host
Host Prefix
Can contain a lowercase letter, number, dot, dash or underscore. Each host label segment must be less than 64 characters, except the first label segment which must be less than 50 characters.
Save changes
Reset

Environment: fr-apim-master-dev

kafka api test 040225 (1.0.0)

Host prefix: myhost .mycompany.org:9092

Configure the Kafka client

To use the Kafka Gateway, you use a regular Kafka client. There are many implementations of the Kafka client, and you can use any client that supports the full Kafka protocol.

As of the 4.6.0 release, the Kafka Gateway requires the Kafka client to be version 3.0 or above.

The default client to talk to Kafka is packaged within the Kafka binary and is based on Java. The prerequisite for using this client is a JRE. See the Java documentation ↗ for more information on how to install a JRE.

1. Download Kafka. Gravitee Kafka Gateway is compatible with the source code or either binary download of each supported Kafka release. For more information about downloading Kafka, go to Kafka's download page ↗.
2. Store the downloaded file structure in a secure place. The root folder is your working directory when calling your Kafka API.

The client is now ready to use. To produce and consume messages, create a `.properties` file in the root folder, as described in the Example section.

At this point, you can begin creating and deploying APIs to the Gravitee Kafka Gateway.

Configure Kafka clients for mTLS authentication

When using an mTLS plan, Kafka clients must present a client certificate during the TLS handshake in addition to the standard TLS configuration. Add the following SSL properties to the client's `.properties` file:

Property	Description	Example
`ssl.keystore.location`	Path to client keystore containing the certificate	`/path/to/client.keystore.jks`
`ssl.keystore.type`	Client keystore type	`JKS`
`ssl.keystore.password`	Client keystore password	`gravitee`

Produce and consume messages

You can use the Kafka Gateway and client to call your Kafka API and, as a primary use case, produce or consume messages. You can also proxy requests to create and manage topics, update partitions, and manage consumer groups.

Prerequisites

The following prerequisites must be met before you can produce and consume Kafka messages:

- You must have an active subscription to a published API plan belonging to your Gravitee Kafka API.
- If you are subscribing to an OAuth2 or JWT plan, your application must reference the same client ID that you use for authorization.

When using Gravitee Access Management (AM) ↗ as the authorization server, the client ID is generated when you create a Gravitee AM Authorization Server resource. To access this resource, you must also create an application in Gravitee Access Management.

For plan, application, subscription, and resource information, see the following:

- For information on how to create and manage plans, see Plans.
- To learn how to create an application for a Gravitee plan, see Applications.
- For more information on how subscriptions work in Gravitee, see Subscriptions.
- To learn how to create a resource, see Resources.

Plan security mutual exclusion

Native Kafka APIs enforce strict separation between plan security types. You can't mix Keyless, mTLS, and authentication plans such as OAuth2, JWT, and API Key in published state. When you publish a plan of one type, all published plans of conflicting types are automatically closed. Multiple plans of the same security type can coexist. For example, two mTLS plans can coexist.

The following table summarizes plan type coexistence rules:

Plan Type	Can Coexist With	Conflicts With
Keyless	Other Keyless plans	mTLS, API Key, OAuth2, JWT
mTLS	Other mTLS plans	Keyless, API Key, OAuth2, JWT
API Key, OAuth2, JWT	Other authentication plans	Keyless, mTLS

If you need to support multiple security types simultaneously, create separate APIs for each security model.

Certificate-based subscription resolution

The gateway extracts the client certificate from the TLS session, computes its MD5 hash, and uses the hash as a security token to look up the subscription. The subscription must be registered in the gateway's trust store manager.

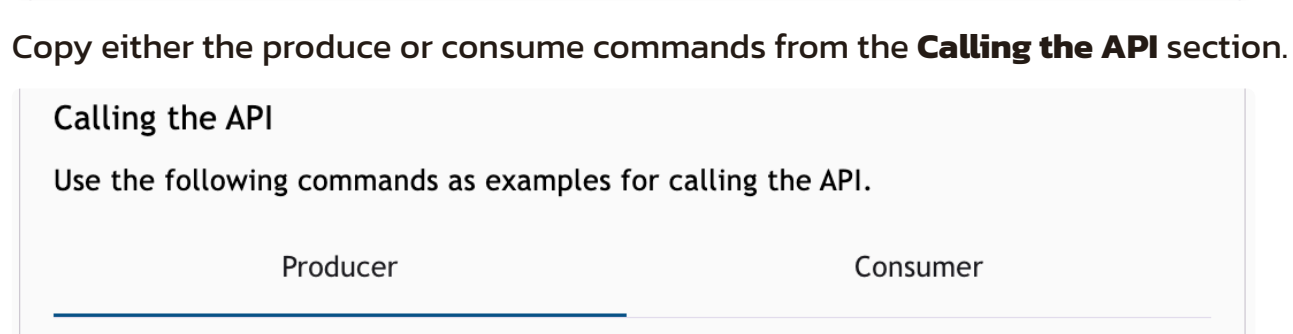
On successful validation, the gateway populates the connection context with the following attributes for metrics and analytics:

- `planId`
- `applicationId`
- `subscriptionId`

Example

The following example provides a template for how to produce and consume messages using the Kafka Gateway, Kafka client, and the prerequisites mentioned above.

1. In the top-level folder of your Kafka download, create an empty `.properties` file named `connect.properties`.
2. Go to the Developer Portal and find your API.
3. After selecting your API, click the **My Subscriptions** tab.
4. Copy the script in the **Review Kafka Properties** section, and then paste it into your `connect.properties` file.

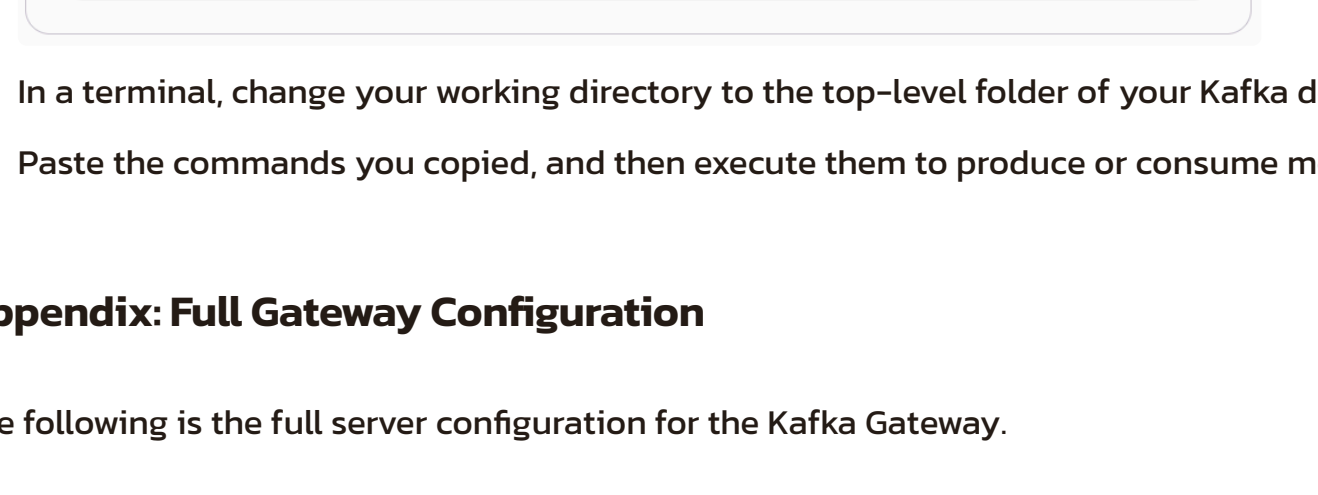


Review Kafka Properties

Create a file `connect.properties` to be used when calling the API.

```
security.protocol=SASL_SSL
sasl.mechanism=OAUTHBEARER
sasl.login.callback.handler.class=org.apache.kafka.common.secu
sasl.oauthbearer.token.endpoint.url={{ TOKEN_ENDPOINT_URL }}
sasl.jaas.config=org.apache.kafka.common.security.oauthbearer.(
  clientId="client_ID" \
  clientSecret="{{ CLIENT_SECRET }}";

# Configure your truststore if you are using a certificate
# not automatically trusted by your installation
# ssl.truststore.location={{ PATH_TO_TRUSTSTORE_JKS }}
# ssl.truststore.password={{ TRUSTSTORE_PASSWORD }}
```

5. Copy either the produce or consume commands from the **Calling the API** section.


Producer
Consumer

Calling the API

Use the following commands as examples for calling the API.

```
./bin/kafka-console-producer.sh \
  --bootstrap-server [redacted]
  --topic test-topic \
  --producer.config connect.properties
```

6. In a terminal, change your working directory to the top-level folder of your Kafka download.
7. Paste the commands you copied, and then execute them to produce or consume messages.

Appendix: Full Gateway Configuration

The following is the full server configuration for the Kafka Gateway.

```
kafka:
  enabled: false

  routingMode: host # default is host. Only host is supported for now.
  # Routing Host Mode
  routingHostMode:
    brokerPrefix: broker-          # default is broker-
    domainSeparator: -             # Used to separate broker's name from api & defaultDomain.

    # The default domain where the Kafka APIs are exposed. ex: `myapi` will be exposed as `my
    defaultDomain: mycompany.org   # Should set according to the public wildcard DNS/Certific
    defaultPort:   9092            # Default public port for Kafka APIs. Default is 9092

    # With the upper default configuration, the Gravitee Kafka gateway yields bootstrap and b
    bootstrapDomainPattern: {apiHost}.mycompany.org
    brokerDomainPattern: broker-{brokerId}-{apiHost}.mycompany.org
    # Where:
    # {apiHost}  is a placeholder that will be replaced when the API is deployed, by the API
    # {brokerId} is a placeholder that stands for the broker id

    # It can be overridden to fit your DNS configuration.
    # Doing so requires BOTH patterns to be set, as well as 'defaultPort'. Please note that '
    # Example:
    #   defaultPort: 9092
    #   bootstrapDomainPattern: bootstrap-{apiHost}.mycompany.org
    #   brokerDomainPattern: {apiHost}-broker{brokerId}.mycompany.org
    #
    #   This configuration yields domains that must target the Gravitee Kafka gateway:
    #      bootstrap-myapi.mycompany.org
    #      myapi-broker0.mycompany.org
    #      myapi-broker1.mycompany.org
    #      ...

  # SSL configuration
  #ssl:
  #  keystore:
      # Supports either JKS or PEM
  #    type: JKS
  #    path: /opt/graviteeio-gateway/ssl/server.keystore.jks
  #    password: secret
      # or
  #    type: PEM
  #    secret: secret://kubernetes/my-certificate
  #    watch: true
  #  openssl: true
  #  # mTLS configuration
  #  clientAuth: required
  #  truststore:
  #    type: jks
  #    password: gravitee
  #    path: /path/to/server.truststore.jks
```